A PThread-based Disk Driver

Due at 16:30 on Monday, 16th March 2026

1 Overview

This document describes the assessed coursework for COMPSCI4011 Operating Systems (H). The goal of this coursework is to implement a piece of code is required which will function as a driver for a simple disk device. Calls will be made to this software component to pass sectors[1] of data to/from the disk. Sectors read from the disk will be passed to appropriate waiting callers. The emphasis is on the concurrency, not on disk scheduling issues. As such, the disk scheduling is FCFS.

The core of your solution will be one or more threads which move sectors of data between queues or buffers (amongst other things). You will be programming in C and using the PThread library. PThreads enable you to dynamically create threads and provides support for both mutual exclusion locks and for condition variables (which support a simple wait/signal mechanism).

Starter "testharness" code is provided on Moodle.

This is an individual assignment.

2 Detailed Problem

You are to write a piece of code which provides the central functionality of a disk device driver (`diskdriver.c`). The ADT you are producing will be called `DiskDriver`, and you will make calls onto an instance of the `DiskDevice` ADT (provided by the test harness). Calls will be made to your code from the application level, as well.

Your code will pass around pointers to `SectorDescriptor`s; this is an ADT about which you need to know almost nothing. One of the components of the system is a `SectorDescriptor` constructor; this will be handed a region of memory (actually a `void *` pointer and a length) and it divides that into pieces, each the right size to hold a `SectorDescriptor`. The `SectorDescriptor`s are then passed to another component, the `FreeSectorDescriptorStore`. The `FreeSectorDescriptorStore` is an unbounded container, which will be used by your code and the test harness as a place from which to acquire `SectorDescriptor`s. When they are no longer used, the `SectorDescriptor`s must be returned to the `FreeSectorDescriptorStore`.

You are allowed to use an existing piece of code in the test harness for the `FreeSectorDescriptorStore`, along with other components of the test harness. Similarly you may use the `BoundedBuffer` provided on Moodle if that is helpful within your solution. *All other code must be your own work; plagiarism detection will be undertaken to check this is the case.*

The test harness includes fake applications, which will acquire `SectorDescriptor`s from the store, initialize their contents and then pass them to your code: the fake applications make two kinds of requests: queue the write of a sector to disk, or queue up a read for a sector from the disk; after making each kind of request, the application makes a subsequent call to either 1) determine that the write was successful, or 2) retrieve the sector after it has been read.

[1] This handout refers to sectors and blocks; they are simply two terms for the same thing, a fixed-size chunk of space on the disk device.

The test harness also includes a fake DiskDevice to which you can pass SectorDescriptors for dispatch. The complication is that the DiskDevice can only write one sector at a time, and is quite slow in doing so; therefore, you may have to queue up other requests. An application thread does not have to wait for its sector to be sent to the disk. Once it has handed it to your code, it is allowed to engage in other activity, returning to your driver to determine that the write has been successful when convenient for the application. This means you are likely to need dedicated threads within your code to pass sectors to the disk device.

Once a sector has been sent to the disk, your code should pass the descriptor back to the FreeSectorDescriptorStore.

The DiskDevice can simultaneously read one sector at a time from the disk. This means that you should always have a read outstanding (if one is available) on the DiskDevice. To maximize the throughput of the DiskDevice, your processing of a recently read sector must be minimal, handing it over to another (buffered?) part of your code for retrieval by the application before initiating another read (if one is available). Again, the application does not wait for its sector to be retrieved from the disk; it will eventually return to the driver to determine that the read has been successful and to access the data read from the disk.

Sectors that have been read will be passed to the fake application layer, and eventually the sector descriptors will be returned to the free sector descriptor store by the applications.

Because this code is supposed to live inside the OS, you are required to use bounded sized data structures within the parts of your code that handle the passing of sector descriptors between the applications and the disk device – i.e. you cannot use malloc(). It's possible that application threads will block, either because they are awaiting incoming sectors, because they are waiting for a write to complete, because the buffers for outgoing/incoming sectors are full (as those data structures are supposed to be bounded), or because there are no free sector descriptors available.

In summary, you have applications that run at their own pace, and a disk device that runs at a probably different pace and for which the requests must be serialized. Your disk driver has to bridge between the two, despite the speed mismatch. You are to use pthreads and bounded data structures to provide this bridging function to implement the spec in diskdriver.h.

The starting files for this exercise are available from Moodle.

It is recommended to begin with a design of your solution, sketching out what you are trying to achieve and the rough code needed to do it. Your sketch will most likely take the form of an interaction and/or collaboration diagram. This diagram is part of the submission for this assessed exercise.

3 Background information (taken from the .h files)

The test harness application threads are distinguished by their PIDs:

```
typedef unsigned long PID;
#define MAX_PID 10
/*
 * A PID is used to distinguish between the different applications
 * It is implemented as an unsigned int, but is guaranteed not to
 * exceed MAX_PID.
 */
```

The actual block number to read or write on the disk is indicated by its Block:

```
/*
```

```
 * Define the data type for a block number on the disk
 */
typedef unsigned long Block;
```

The disk device is "full duplex" – i.e., it can write a sector and read a sector at the same time. Writing a sector entails the calling thread simply invoking a method in the device, with that thread blocking until the sector has been successfully written or until an error is detected. Reading a sector entails making a different call to the device, with that thread blocking until the sector has been successfully read or until an error is detected.

A `SectorDescripter` contains the `PID` of the requesting application, the `Block` of the sector of interest, and the buffer containing the data to write or a buffer into which the sector data is read.

The disk device supports the following calls – your code may invoke these:

```
/*
 * write sector to disk, returns 1 if successful, 0 if unsuccessful
 * this may take a substantial amount of time to return
 * if unsuccessful, you should return an indication of this lack of
 * success to the application
 */
int write_sector(DiskDevice *dd, SectorDescriptor *sd);

/*
 * read sector from disk, returns 1 if successful, 0 if unsuccessful
 * this may take a substantial amount of time to return
 * if unsuccessful, you should return an indication of this lack of
 * success to the application
 */
int read_sector(DiskDevice *dd, SectorDescriptor *sd);
```

The fake application threads may make the following calls to your code – you must implement them.

```
/*
 * the following calls are used to write a sector to the disk;
 * the nonblocking call must return promptly, returning 1 if successful at
 * queueing up the write, 0 if not (in case internal buffers are full)
 * the blocking call will usually return promptly, but there may be
 * a delay while it waits for space in your buffers.
 * neither call should delay until the sector is actually written to the disk
 * for a successful nonblocking call and for the blocking call, a voucher is
 * returned that is required to determine the success/failure of the write
 */
void blocking_write_sector(SectorDescriptor *sd, Voucher **v);
int nonblocking_write_sector(SectorDescriptor *sd, Voucher **v);

/*
 * the following calls are used to initiate the read of a sector from the disk
 * the nonblocking call must return promptly, returning 1 if successful at
 * queueing up the read, 0 if not (in case internal buffers are full)
```

```
 * the blocking call will usually return promptly, but there may be
 * a delay while it waits for space in your buffers.
 * neither call should delay until the sector is actually read from the disk
 * for a successful nonblocking call and for the blocking call, a voucher is
 * returned that is required to collect the sector after the read completes.
 */
void blocking_read_sector(SectorDescriptor *sd, Voucher **v);
int nonblocking_read_sector(SectorDescriptor *sd, Voucher **v);

/*
 * the following call is used to retrieve the status of the read or write
 * the return value is 1 if successful, 0 if not
 * the calling application is blocked until the read/write has completed
 * if a successful read, the associated SectorDescriptor is returned in *sd
 */
int redeem_voucher(Voucher *v, SectorDescriptor **sd);
```

There are also some initialization calls, the first is for you to implement, the second is one you can use:

```
void init_disk_driver( DiskDevice *dd, void * mem_start,
        unsigned long mem_length, FreeSectorDescriptorStore **fsds);
/*
 * Called before any other methods[2], to allow you to initialize
 * data structures and start any internal threads.
 * Arguments:
 *      dd: the DiskDevice that you must drive,
 *      mem_start, mem_length: some memory for SectorDescriptors
 *      fpds: You hand back a FreeSectorDescriptorStore into
 *      which SectorDescriptors built from the memory
 *      described in mem_start and mem_length have been placed
 */

void init_sector_descriptor(SectorDescriptor *sd);
/*
 * Resets the sector descriptor to be empty.  Used by fake applications
*/
```

You will need to use the `FreeSectorDescriptorStore` facilities. You can create a `FreeSectorDescriptorStore` and populate it with Sector`Descriptor`s created from a memory range using:

```
FreeSectorDescriptorStore *FreeSectorDescriptorStore_create(
                void *mem_start, unsigned long mem_length);
```

and then acquire and release `SectorDescriptor`s using:

```
void blocking_get_sd(FreeSectorDescriptorStore *fsds, SectorDescriptor **sd);
int nonblocking_get_sd(FreeSectorDescriptorStore *fsds, SectorDescriptor **sd);
```

[2] When an operating system boots up, it typically invokes an initialization routine in each kernel module to enable that module to prepare for operation. It invokes these initialization routines before it enables any applications to be launched.

```
void blocking_put_sd(FreeSectorDescriptorStore *fsds, SectorDescriptor *sd);
int nonblocking_put_sd(FreeSectorDescriptorStore*fsds, SectorDescriptor*sd);

/*
 * As usual, the blocking versions only return when they succeed.
 * The nonblocking versions return 1 if they worked, 0 otherwise.
 * The _get_ functions set their final arg if they succeed.
 */
```

Finally, given a SectorDescriptor, you can access the embedded PID and Block fields using:

```
PID sector_descriptor_get_pid(SectorDescriptor *sd);
Block sector_descriptor_get_block(SectorDescriptor *sd);
```

4 Developing Your Code

The best way to develop your code is in a vanilla Linux environment, this could be on a lab machine, a virtual machine image, or on your laptop. I would recommend a VM image, this way, if you crash the system, it is straightforward to restart. This also gives you the benefit of taking snapshots of system state right before you do something potentially risky or hazardous, so that if something goes horribly awry you can easily roll back to a safe state.

The starter code has been compiled on an x86_64 machine on Linux, as found in VM on Moodle. This is also where your submissions will be assessed. Other architectures are not supported.

You should use a code repository to keep track of your programming efforts.

5 Testing your driver

The Makefile provided builds a program named my_demo from your diskdriver.o and the rest of the object files provided in the starting archive. A working version of the testharness built from a correct disk driver is also in the archive, named demo. Both demo and my_demo take a single, optional argument – the number of seconds to run the simulation. If you do not specify the optional argument, then the simulation will run for 2 minutes (120 seconds). I suggest that while testing your diskdriver implementation, you use the following command line to bash:

```
$ ./my_demo 30 2>&1 | tee log
```

This will cause your simulation to run for 30 seconds, and cause everything written to stdout or stderr to be piped to tee, which will save all such output to a file named log, as well as display it on your screen while it is working.

The previous sections have described constraints on the workings of your driver. Here I list diagnostics that will be displayed by the simulation that indicate that you have violated one or more of these constraints.

BUG: Two or more parallel calls to write a sector
The diskdevice can only handle one write at a time; you have asked it to do 2 or more.

BUG: Two or more parallel calls to read a sector
The diskdevice can only handle one read at a time; you have asked it to do 2 or more.

6 Submission

The following describes what to submit and how to submit it. Not following submission guidelines or late submissions will result in losing marks and/or being penalised according to university policy.

- You must submit your solutions electronically by uploading a zipped (.zip) archive via Moodle.
- Your archive must be named <your_matric_number + first_letter_of_family_name>. This usually corresponds to your university email address before the '@'. For example, 0123456h.zip.
- Your archive **must** contain
    - your "diskdriver.c" code containing your solution
    - a document named "report.pdf" or "report.txt", describing the state of your solution, and documenting anything of which we should be aware when marking your submission.
- Your archive *may* contain
    - Section 2 above indicates that you should prepare a sketch of your design; the most useful form of such a sketch is as an interaction and/or a collaboration diagram. If you submit a PDF file for your report, you should include the diagram in your report; if you submit a TXT file for your report, you should include design.jpg in your submitted archive. In either case, it can be hand-drawn diagram that is then scanned to a JPEG file.
- Beyond the above, there should be no other files in your archive.
- Your archive should ***not*** be contained in a folder -> please be aware of default OS settings. Thus, if you upload "0123456h.tgz", then we should see the following when we execute the following command:

```
% unzip 0123456h.zip
-rw-rw-r-- harveyp/group  5125 2025-03-30 16:37 diskdriver.c
-rw-rw-r-- harveyp /group   527 2025-03-30 16:30 report.txt
-rw-rw-r-- harveyp /group 12345 2025-03-30 16:24 design.jpg
```

Each of your source files must start with an "authorship statement", contained in C comments, as follows:

- state your name, your GUID, and the title of the assignment (COMPSCI4011 – Assessed Exercise)
- state either "This is my own work." or "This is my own work except that …", as appropriate.